



PROCESADORES DE LENGUAJE - INICIO TRADUCTOR

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

¿Qué elementos léxicos proporciona rd_lex()?

La función `rd_lex()` proporciona los siguientes elementos léxicos en el análisis de la entrada:

1. **Números:** Se detectan como secuencias de uno o más dígitos consecutivos. Se almacenan en `tokens.number` y se devuelve el token `T_NUMBER`.
2. **Variables:** Se reconocen como una sola letra (A, x, y) o una letra seguida de un único dígito (A1, x2). Se almacenan en `tokens.variable_name` y se devuelve el token `T_VARIABLE`.
3. **Operadores aritméticos:** Se identifican los operadores +, -, * y /. Se almacenan en `tokens.token_val` y se devuelve el token `T_OPERATOR`.
4. **Caracteres especiales y literales:** Cualquier otro carácter que no sea un número, variable u operador se devuelve directamente como su código ASCII. Esto incluye los paréntesis (y), el signo de igualdad =, el salto de línea \n y otros caracteres.
5. **Fin de archivo (EOF):** Si se encuentra el final del archivo, el programa finaliza.

Ejemplo de análisis léxico para la expresión `(+ A (* 3 4))`:

- (→ Se devuelve el carácter '(' como token.
- + → Se reconoce como operador y se devuelve `T_OPERATOR`.
- A → Se reconoce como variable y se devuelve `T_VARIABLE`.
- (→ Se devuelve el carácter '(' como token.
- * → Se reconoce como operador y se devuelve `T_OPERATOR`.
- 3 → Se reconoce como número y se devuelve `T_NUMBER`.
- 4 → Se reconoce como número y se devuelve `T_NUMBER`.
-) → Se devuelve el carácter ')' como token.
-) → Se devuelve el carácter ')' como token.

La gramática que hemos desarrollado es la siguiente:

GRAMÁTICA INICIAL:

`<Programa> ::= <Expresion> "\n"`

`<Expresion> ::= <Numero>`

`| <Variable>`

`| "(" <Operador> <Expresion> <Expresion> ")"`

`| "(" "=" <Variable> <Expresion> ")"`

`<Operador> ::= "+" | "-" | "*" | "/"`

`<Numero> ::= <Digito> | <Numero> <Digito>`

`<Variable> ::= <Letra> | <Letra> <Digito>`

`<Digito> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"`

<Letra> ::= "A" | "B" | "C" | ... | "Z" | "a" | "b" | "c" | ... | "z"

=====

GRAMATICA FINAL: (LL1)

<Programa> ::= <Expresion> "\n"

<Expresion> ::= <Numero>

| <Variable>

| "(" <InicioExpresion> ")"

<InicioExpresion> ::= <Operador> <Expresion> <Expresion>

| "=" <Variable> <Expresion>

<Operador> ::= "+" | "-" | "*" | "/"

<Numero> ::= <Digito>

<Variable> ::= <Letra>

<Digito> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

<Letra> ::= "A" | "B" | "C" | ... | "Z" | "a" | "b" | "c" | ... | "z"

En esta última hemos quitado la recursividad a izquierdas, además de que dos expresiones no empiecen por el mismo término.

Hemos introducido la gramática en jflap y nos ha quedado el resultado que se ve en la imagen posterior. Como los número y las variables se analizan en el léxico no se introducen y en el jflap simplemente dejamos la parte del sintáctico, esto hace que se cumpla que sea una gramática LL(1)

JFLAP: <untitled1>

FileInputTestConvertHelp

EditorBuild LL(1) ParseLL(1) Parsing

Table Text Size

	(	)	*	+	-	/	=	d	n	v	\$
E	(I)			OEE	OEE	OEE	OEE	=vE	d		
I			*	+	-	/					
O											
P	En							En		En	

StartStep

Noninverted Tree

Input(+dd)n

Input Remaining\$

Stack

Input Field Text Size (For optimization, move one of the window size adjustors around this window after resizing the text fields)

Table Text Size

LHS		RHS
P	→	En
E	→	d
E	→	v
E	→	(I)
I	→	OEE
I	→	=vE
O	→	+
O	→	-
O	→	*
O	→	/

String successfully parsed!